

Implementation of an AXI-compliant lock-in amplifier on the RedPitaya open source instrument

L. H. Arnaldi

Laboratorio Detección de Partículas y Radiación
Centro Atómico Bariloche - Instituto Balseiro
Bariloche, Rio Negro (8400)
Email: arnaldi@cab.cnea.gov.ar

Abstract—This paper describes the implementation of a lock-in amplifier (LIA) which conforms to the AXI standard for on-chip communication. The design and implementation is based on the Zynq[®] field programmable gate array (FPGA) present in the open-source instrument RedPitaya. The designed architecture is a mixed solution between VHDL hardware modules and software modules, running within an ARM CPU. General design criteria are summarized for the all-digital implementation. Simulation results for the performance of some of the designed intellectual property (IP) blocks are also presented to verify its operation.

Index Terms—Phase-sensitive detection, FPGA, Zynq, RedPitaya, all-digital phase locked loop.

I. INTRODUCTION

Lock-in amplifiers are measurement instruments widely used in science and engineering. They are powerful instruments to detect and measure periodic signals in presence of a large quantity of noise. The LIA use a technique known as phase-sensitive detection to single out a component of interest from a noisy signal, at a specific reference frequency and phase, attenuating the rest of the spectrum. The output is a DC component proportional to the signal amplitude at the frequency of interest. The functional description of this technique is well known and is widely addressed in literature [1]–[3].

For a long time lock-in amplifiers were based on analog electronics. However, digital LIAs have become increasingly popular in many experimental setups due to the combination of flexibility, cost and performance. If sufficient precision is used in all calculations, the linear response range of each digitized component can be arbitrarily large, making the design process easier. Also, using digital multipliers, almost ideal digital mixers and filters can be created.

Recently, several FPGA-based lock-in amplifiers for different applications (including one with sub-ppm resolution) have been reported [4]–[7]. Advantages of FPGA-based LIA systems were investigated in [5] and [6], where emphasis was given to show the FPGA's flexibility in fulfilling more than a single digital task at the same time and aiming to obtain great flexibility on key design parameters, such as sampling and lock-in frequency. In [5] there is an implementation using a Zynq[®] FPGA, tailored to a low-frequency application. None of these publications, however, presents a procedure for designing an all-digital LIA from scratch, tailored to a RedPitaya board, using the AXI standard and showing the details of the calculations, as it is intended in this paper.

The implementation of the lock-in amplifier used is the well-known dual-phase LIA [6]. Details are given for the design and construction of the FPGA-based LIA along with some testing of the final product.

The analog signals are acquired using a dual-channel, 14-bit A/D converter and mixed with a pair of quadrature reference signals. The reference signal can be derived from an external source or generated internally by a programmable direct digital synthesis module. The external reference signal is sampled and then converted to a pulse train by a zero crossing detector module. This pulse train feeds the reference input of an all-digital phase locked loop (ADPLL). The ADPLL generates the required quadrature sine waves. Once mixed down, the signals are fed through programmable digital filters to a dual-core ARM CPU for the determination of the amplitude and phase of the input signal. The output signals can be directly routed to the analog output ports, from the ARM CPU. The ARM processor acts as a high level control system allowing to control the filters operation for example, giving the advantage of maximal flexibility, as well as a useful tool in debugging.

Constructing these modules in an FPGA allows all operations after A/D conversion to be done digitally at high speed. The target platform for this system is a Zynq[®] (xc7z010c1g400-1) FPGA, present in the RedPitaya open-source board, but the design concepts can be easily adapted to another platforms. As a reasonable expectation, input signal sine waves operating at frequencies of up to some MHz should be easily analyzed at the ADC's clock rate of 125 MHz.

The following sections describe the design of the digital electronics of the realized instrument. Section II gives an overview of the AXI standard. In Section III the main characteristics of the RedPitaya open-source instrument are presented. In Section IV, a design flow for the LIA is described. Section V presents some simulations of the designed blocks. Finally, conclusions are provided in Section VI.

II. THE AXI STANDARD

AXI stands for Advanced eXtensible Interface, and the current version is AXI4, which is part of the ARM AMBA[®] 3.0 open standard. Many devices and IP blocks produced by third party manufacturers and developers are based on this standard.

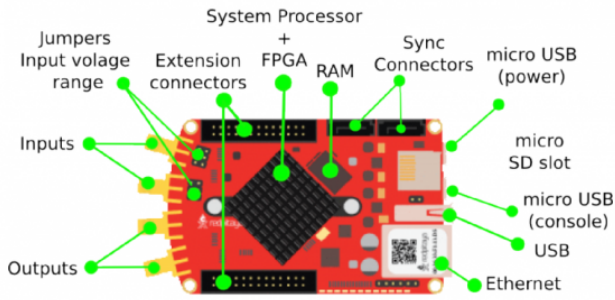


Fig. 1. RedPitaya board hardware overview. (Source <https://wiki.redpitaya.com/>)

The AMBA standard was originally developed by ARM for use in microcontrollers, with the first version being released in 1996. Since then, the standard has been revised and extended, and it is now described by ARM as “the *de facto* standard for on-chip communication” [8]. The focus is now on System-on-Chip (SoC), including SoCs based on FPGAs or, in the case of Zynq®, a device which includes FPGA fabric [9].

Nowadays there are three flavours of AXI4, each of which represents a different bus protocol, as summarised below. The choice of AXI bus protocol for a particular connection depends on the desired properties of that connection.

- AXI4 [10] : For memory-mapped links, and providing the highest performance: an address is supplied followed by a data burst transfer of up to 256 data words (or “data beats”).
- AXI4-Lite [10] : A simplified link supporting only one data transfer per connection (no bursts). AXI4-Lite is also memory-mapped: in this case an address and single data word are transferred.
- AXI4-Stream [11] : For high-speed streaming data, supporting burst transfers of unrestricted size. There is no address mechanism; this bus type is best suited to direct data flow between source and destination (non memory mapped).

For this case then, the best choice is to use the AXI4-Stream protocol in the LIA implementation, due to its inherent capacity for high-speed data transmission.

III. THE REDPITAYA BOARD

Fig. 1 shows an overview of the RedPitaya board. RedPitaya is a low-cost (~ 400USD) analog signal generation/measurement electronics [12]. Its specifications are shown in Table I. The RedPitaya is equipped with dual fast ADC and dual fast DAC, i.e., it is easy to acquire and generate two sine waves of arbitrary phase difference simultaneously. These are the important specifications for its usage as LIA. It runs a Linux on it and can be controlled/accessed via several ways: browsers via PC or tablet (it can run a web server), usb-serial console, and SSH protocol. The user can easily write/modify source code to control the FPGA and CPU by using open-source software. For example, it is possible to implement own

TABLE I
SPECIFICATIONS OF REDPITAYA

Processor	Xilinx Zynq-7010 SoC ARM dual core CPU and Artix 7 FPGA
System disk	MicroSD
RAM	512MB
Access	USB console, Ethernet, WiFi dongle
Power	5V x 2A max., 0.9A typ.
Fast DAC	Dual channel, 14-bit, 125MSPS
Fast ADC	Dual channel, 14-bit, 125MSPS
Other I/Os	Slow DAC x 4, slow ADC x 4, digital I/O x 16, daisy chain connector

modules on FPGA by using VHDL or Verilog, and run C programs on the ARM CPU. Various open-source applications are also available.

IV. SYSTEM ARCHITECTURE

The implemented system follows the Xilinx paradigm in constructing the new designs like interconnected intellectual properties blocks, so each module in this design was programmed and constructed like an intellectual property block. The modules were programmed using the Xilinx Vivado® EDA software [13]. There is a top level module, called *system_wrapper.vhd*, which define all other supporting VHDL modules. These modules are analysed in detail in this section. All the IP blocks conforms to the AXI4-Stream standard, which is best suited for this application. The top level implementation of the LIA can be seen in Fig. 2 as a block diagram.

A. ADC and DAC interfaces

These modules act as interfaces between the FPGA and the external world. They read and write data to the external integrated circuits through a 14-bit bus. The design of the IP blocks is targeted to the DAC1401D125 IC for the D/A conversion and to the LTC2145-14 IC for the A/D conversion.

B. Zero crossing detector (ZCD)

This module takes the digitized reference input and waits for zero crossing. In order to prevent noise in the triggering signal, an hysteresis was added for detecting both positive and negative going zero crossing. To prevent any phase shifts, the output is always triggered at the zero crossing. The external reference is then converted to a pulse train to feed the reference input of the all-digital phase locked loop block.

C. All-digital phase locked loop

A phase locked loop (PLL) is needed in order to lock to an externally generated reference signal. The implementation of this module follows an all-digital phase locked loop architecture. The input signal is the digitized data stream of the reference signal and the output is a pair of locked quadrature sine waves. It can also use an internally generated reference signal, provided by a programmable direct digital synthesis

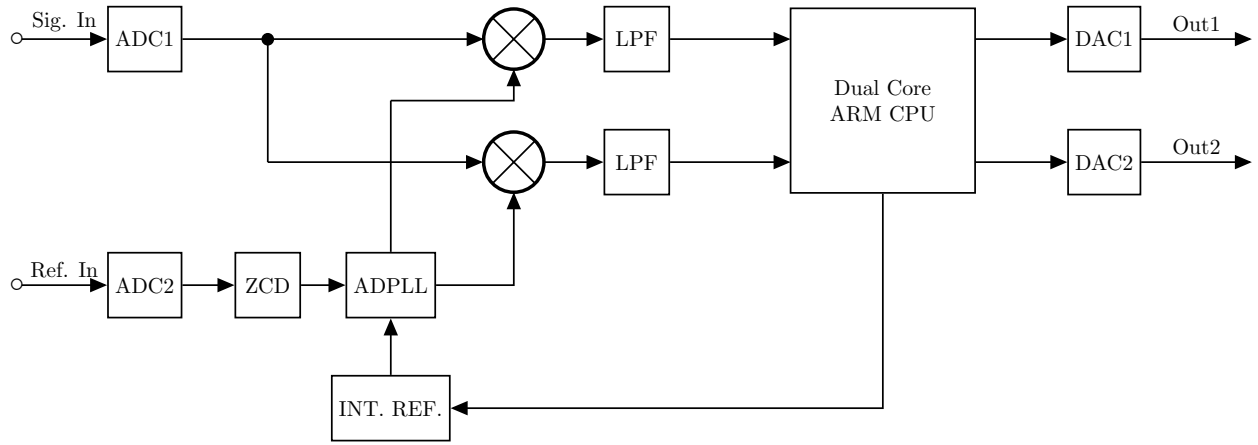


Fig. 2. Block diagram of the implemented LIA. It is composed of two A/D interfaces, a zero crossing detector (ZCD), an all-digital phase locked loop (ADPLL), a programmable direct digital synthesis unit (INT. REF.), two signed multipliers, two recursive low-pass filters (LPF), the dual core ARM CPU and two D/A interfaces.

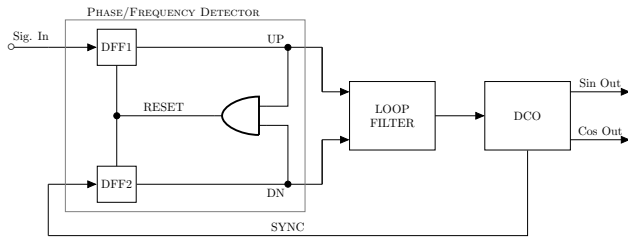


Fig. 3. ADPLL. As normal PLL the building blocks of the ADPLL are phase detector, loop filter, digital-controlled oscillator (DCO), and optional divide by N element.

block (INT. REF. in Fig. 2), which can be activated/deactivated by the processor.

The presented PLL can be approximated with a model of a continuous PLL. This simplifies the calculation of some generic PLL parameters. The chosen topology for the ADPLL can be seen in Fig. 3. The building blocks include a phase detector, a loop filter, a digital-controlled oscillator (DCO), and optionally a divide by N element (not shown in Fig. 3).

For simplicity, both design and conceptual, this document is focused on a design approach for type-II second-order all-digital phase locked loop [14], [15]. Details of the necessary calculations are given in [16] and [17]. In general, the approach showed here can be extended for the design of all-digital PLLs with different types and orders.

1) *Phase/Frequency sensitive detector*: A phase detector is a circuit capable of delivering an output signal that is proportional to the phase difference between its two input signals. In this design, a digital phase/frequency detector (PFD) is implemented. The labeled box in Fig. 3 shows the schematic of the PFD. Because the output signal of the PFD depends on phase error in the locked state of the PLL and on the frequency error in the unlocked state, a PLL that uses the PFD will lock under any condition, irrespective of the type

of loop filter used. For this reason the PFD is the preferred phase detector in PLLs [18]. The PFD is built from two D-type flip flops, whose outputs are denoted UP and DN (down), respectively. The PFD can be in one of three states:

- UP = 0, DN = 1 → State = -1
- UP = 0, DN = 0 → State = 0
- UP = 1, DN = 0 → State = 1

The actual state of the PFD is determined by the positive transition of the signals SYNC and Sig. In as shown in Fig. 3.

2) *Loop filter*: The loop filter acts upon the output of the phase detector to remove unwanted high frequency terms, and produce the signal that drives the DCO. The design of the loop filter is vital in defining the overall characteristics and behaviour of the PLL.

A digital equivalent of an analog second-order loop filter consists of a proportional path with a gain K_P and an integral path with a gain K_I . It is called a *Proportional + Integral* (PI) or *Lead-Lag* filter. Fig. 4 shows a schematic representation of the loop filter. The parameters of a digital loop filter K_P and K_I can be obtained from the parameters of an analog loop filter by using the bilinear transform (1)

$$s = \frac{2}{T_s} \frac{1 - z^{-1}}{1 + z^{-1}} \quad (1)$$

where T_s is the sampling time of the discrete system.

Eq. (2) gives the s-domain transfer function of the PI loop filter and eq. (3) is the transfer function after the application of the transformation (1):

$$\frac{u(s)}{e(s)} = K_P + \frac{K_I}{s} \quad (2)$$

$$\frac{u(z)}{e(z)} = K_P + K_I \frac{1}{1 - z^{-1}} \quad (3)$$

where K_P and K_I are the only unknown parameters that need to be determined for the ADPLL design at this stage.

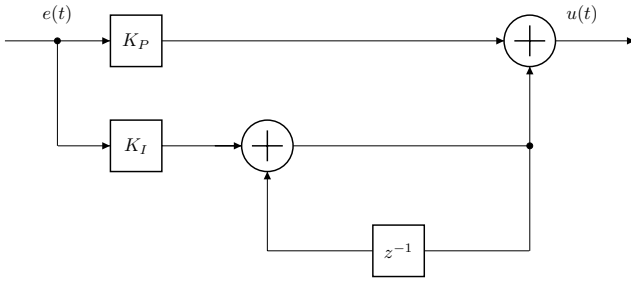


Fig. 4. Building blocks of the loop filter, it is a proportional + integral (PI) filter. The proportional output is $+K_P$ or $-K_P$ and the integral output is the accumulation of $+K_I$ or $-K_I$ over time.

Since $e(t)$ takes a value of either $+1$ or -1 , the proportional output will be $+K_P$ or $-K_P$. On the other hand, the integral output is the accumulation of $+K_I$ or $-K_I$ over time. This filter has one pole on the unit circle.

3) *Digital-controlled oscillator*: The programmable oscillator is implemented as an high accuracy digital-controlled oscillator (DCO). The loop filter provides a frequency tuning word (FTW) at the input of the DCO. The FTW is a fixed point 32-bit digital word, in 24.8 format, setting the working frequency. The DCO generates a pair of quadrature sine waves (14-bit resolution) to feed the mixers and a digital square wave of the same frequency feeding the phase comparator (*SYNC* signal in Fig. 3).

The output frequency of the DCO can be calculated as:

$$f_{out} = \frac{A f_{clk} FTW}{2^{N-P}} \quad (4)$$

where f_{clk} is the system clock, FTW is the frequency tuning word output from the loop filter, N is the word length of the phase accumulator ($N = 50$ in this case), P is the number of bits after the decimal point and A is a proportional factor obtained from:

$$\frac{A f_{clk}}{2^{N-P}} = 1 \quad (5)$$

The frequency resolution of this module, for 8-bit after the decimal point is:

$$\Delta f = \frac{1}{2^8} = 3.9 \times 10^{-3} \text{ Hz} \quad (6)$$

D. Low-pass filters

The low-pass filters were designed to emulate a simple analog RC circuit with adjustable time constant. The modules were implemented as single pole recursive filters, allowing easy on-the-fly calculation of filter parameters when changing settings.

The single pole filter output is given by:

$$y[n] = a_0 x[n] + b_1 y[n-1] \quad (7)$$

The coefficients are found from these simple equations:

$$a_0 = 1 - x, \quad (8a)$$

$$b_1 = x \quad (8b)$$

The characteristics of the filter are controlled by the parameter, x , a value between zero and one. Physically, x is the amount of decay between adjacent samples. The higher the value of x , the slower the decay.

The value for x can be directly specified, or found from the desired time constant of the filter. Just as $R \times C$ is the number of seconds it takes an RC circuit to decay to 36.8% of its final value, d is the number of samples it takes for a recursive filter to decay to this same level:

$$x = e^{-1/d} \quad (9)$$

For instance, a sample-to-sample decay of $x = 0.86$ corresponds to a time constant of $d = 6.63$ samples. There is also a fixed relationship between x and the -3 dB cutoff frequency, f_c , of the digital filter:

$$x = e^{-2\pi f_c} \quad (10)$$

This provides three ways to find the “a” and “b” coefficients, starting with the time constant, the cutoff frequency, or just directly picking x [19]. The parameters can be generated by the ARM CPU from a user selected time constant.

E. Lock indicator

Although not shown in the schematics, a lock indicator was also implemented. The PFD lends itself to an effective and simple scheme, as described below. A two-input OR gate takes as its inputs the UP and DN outputs of the PFD. When the PLL is locked with small phase error, neither UP nor DN is true for any but very short intervals during each comparison cycle. When the PLL is out of lock, either UP or DN will be true, on average over many cycles, for 50% or more of the time. The basis of lock detection is to pass the output of the OR gate through a smoothing filter to extract its average dwell time in the true state and to compare that average against a suitable threshold (say, 25% average true dwell time). The PLL is deemed to be locked if the average true time is below the threshold and unlocked if the average true time is above the threshold [14]. The lock detector also include a timer that requires the lock indication to persist for a specified time interval before phase lock is declared. The timer is started when the average dwell time falls below threshold and reset to zero whenever the threshold is exceeded before the timer reaches its specified interval.

V. SIMULATIONS

Simulations were carried in order to test the behaviour of some of the blocks of the system. Fig. 5 shows the output of the DCO when it is applied a $FTW = (2560000000)_{10}$, corresponding to a generated output frequency of $f = 10 \text{ MHz}$. In the same figure can be seen the two outputs I and Q (labeled

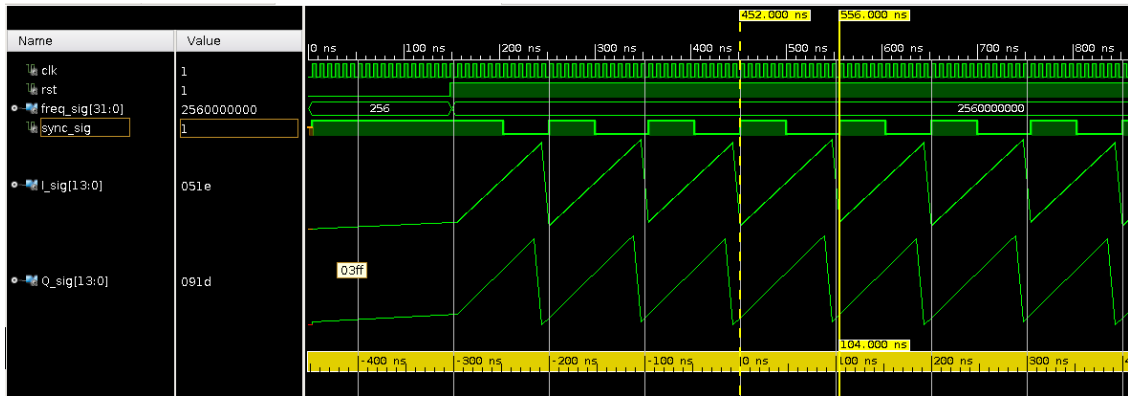


Fig. 5. Simulation of the output of the DCO. The FTW input is $(2560000000)_{10}$, giving a frequency of 10 MHz at the output.

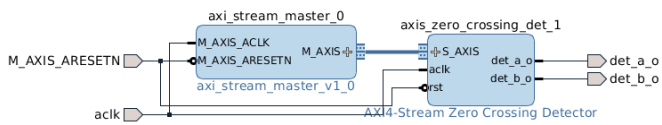


Fig. 6. Block design to test the ZCD module as seen in Vivado Design Suite. The module was designed taking into account the two ADC's of the RedPitaya board, hence the two outputs *det_a_o* and *det_b_o*.

I_sig and Q_sig respectively) and the $sync_sig$ signal ($SYNC$ signal in Fig. 3).

An arrangement was programmed to test the compliance of the modules with the AXI4-Stream standard. An AXI master module was coded to manage the interface and test the different blocks. As can be seen from Fig. 6, the ZCD module has the necessary interface signals to conform to the AXI4-Stream standard. The same test were carried for all the modules in this design. As an example, Fig. 7 shows the simulation of the AXI interface for the ZCD module.

Fig. 8 shows the behaviour of the phase detector output signals (UP and DN) when the input signal is in phase with the reference ($SYNC$) from the DCO and Fig. 9 shows the same output signals when they are not in a lock condition.

VI. CONCLUSIONS

The design of an highly configurable, low-cost, FPGA-based lock-in amplifier has been presented in this work. It was designed from scratch aiming to provide a flexible platform to use in different experiments.

The designed IP blocks conforms to the AXI standard, the *de facto* standard for on-chip communication. The behaviour of the modules were tested by simulations, showing good agreement with the standard.

The implementation of the system showed to be highly configurable and flexible, thanks to the use of the Zynq[®] system on chip (SoC) from Xilinx.

There are still future work in the characterization of the system performance through the use in-place with some experiments.

The proposed LIA architecture is simple, can be easily replicated following the design guidelines showed here, so resulting particularly suitable for its use as an educational tool in courses of FPGA-based systems. With respect to conventional LIAs typically working at low operating frequencies, the developed circuit is capable to manage high frequency signals, of the order of MHz.

REFERENCES

- [1] S. R. Systems, “About Lock-In Amplifiers,” dec 1992.
- [2] P. Horowitz and W. Hill, *The Art of Electronics*. Cambridge University Press, mar 2006, vol. 1.
- [3] R. E. Simpson, *Introductory Electronics for Scientists and Engineers*. Addison-Wesley, abr 1987.
- [4] G. Gervasoni, M. Carminati, and G. Ferrari, “FPGA-based lock-in amplifier with sub-ppm resolution working up to 6 MHz,” *2016 IEEE Int. Conf. Electron. Circuits Syst. ICECS 2016*, pp. 117–120, 2017.
- [5] G. Giacomia, G. Greco, L. Mistretta, and R. Rizzo, “Exploring FPGA-Based Lock-In Techniques for Brain Monitoring Applications,” *Electronics*, vol. 6, no. 1, p. 18, 2017. [Online]. Available: <http://www.mdpi.com/2079-9292/6/1/18>
- [6] G. Macias-Bobadilla, J. Rodríguez-Reséndiz, G. Mota-Valtierra, G. Soto-Zarazúa, M. Méndez-Loyola, and M. Garduño Aparicio, “Dual-phase lock-in amplifier based on FPGA for low-frequencies experiments,” *Sensors (Switzerland)*, vol. 16, no. 3, 2016.
- [7] A. Restelli, R. Abbiati, and A. Geraci, “Digital field programmable gate array-based lock-in amplifier for high-performance photon counting applications,” *Rev. Sci. Instrum.*, vol. 76, no. 9, 2005.
- [8] ARM, “AMBA Open Specification.” [Online]. Available: <http://www.arm.com/products/system-ip/amba/amba-open-specifications.php>
- [9] L. H. Crockett, R. A. Elliot, and M. A. Enderwitz, *The Zynq Book Tutorials for Zybo and Zedboard*. Strathclyde Academic Media, ago 2015.
- [10] ARM, “AMBA AXI and ACE Protocol Specification: AXI3, AXI4, and AXI-Lite, ACE and ACE-Lite.” [Online]. Available: <http://www.arm.com/products/system-ip/amba/>
- [11] —, “AMBA 4 AXI4-Stream Protocol Specification.” [Online]. Available: <http://www.arm.com/products/system-ip/amba/>
- [12] RedPitaya, “RedPitaya Open Source Instrument.” [Online]. Available: <http://www.redpitaya.com/>
- [13] Xilinx Inc., “Vivado Design Suite.” [Online]. Available: <https://www.xilinx.com/products/design-tools/vivado.html>
- [14] F. M. Gardner, *Phaselock Techniques*. Wiley, jul 2005, vol. 1.
- [15] M. Rice, *Digital Communications: A Discrete-Time Approach*. Prentice Hall, jul 2009.
- [16] V. Kratyuk, P. K. Hanumolu, U.-K. Moon, and K. Mayaram, “A Design Procedure for All-Digital Phase-Locked Loops Based on a Charge-Pump Phase-Locked-Loop Analogy,” *IEEE Trans. Circuits Syst. II Express Briefs*, vol. 54, no. 3, pp. 247–251, mar 2007. [Online]. Available: <http://ieeexplore.ieee.org/document/4132962/>

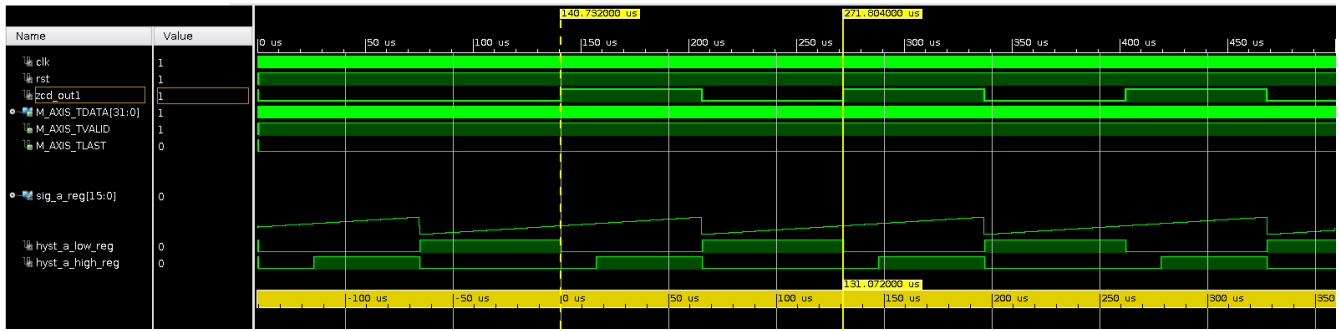


Fig. 7. Simulation of the output of the ZCD module and its AXI-Stream interface. The output is a signal of $f = 7629.3$ Hz (labeled *zcd_out1*). Can be seen the two registers holding the high and low hysteresis values (*hyst_a_high_reg* and *hyst_a_low_reg*, respectively). The block complies with the AXI standard.

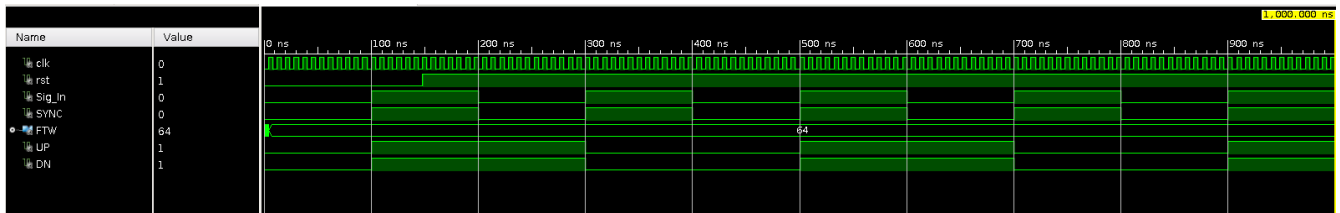


Fig. 8. Output of the phase detector, UP and DN signals. The input signal, *Sig_in* is locked to the reference, *SYNC*.

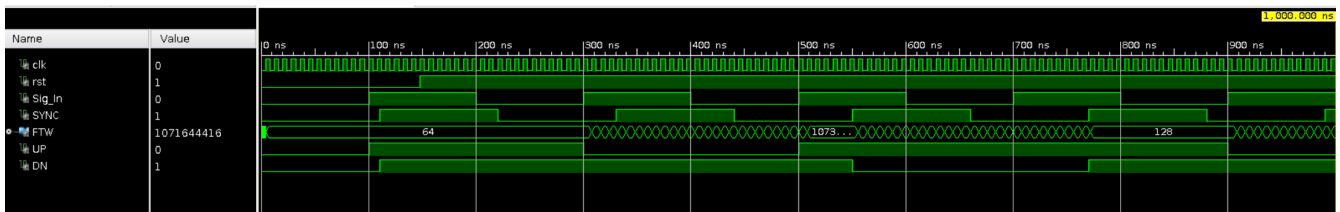


Fig. 9. Output of the phase detector, UP and DN signals. The input signal is not locked to the reference.

- [17] M. Kumm, H. Klingbeil, and P. Zipf, "An FPGA-Based Linear All-Digital Phase-Locked Loop," *IEEE Trans. Circuits Syst. I Regul. Pap.*, vol. 57, no. 9, pp. 2487–2497, sep 2010. [Online]. Available: <http://ieeexplore.ieee.org/document/5456240/>
- [18] R. E. Best, *Phase-Locked Loops*. McGraw Hill Professional, jun 2003, vol. 1.
- [19] S. W. Smith, *Digital Signal Processing: A Practical Guide for Engineers and Scientists*. Newnes, feb 2003, vol. 1.